

```

1  #include <stdbool.h>
2  #include <stdio.h>
3
4  #define MAX 100
5
6  int max[MAX][MAX];
7  int allocation[MAX][MAX];
8  int need[MAX][MAX];
9  int available[MAX];
10
11 int numProcesses, numResources;
12
13 void input();
14 void calculateNeed();
15 void display();
16 void bankersAlgorithm();
17
18 int main() {
19     printf("***** Banker's Algorithm *****\n");
20
21     input();
22     calculateNeed();
23     display();
24     bankersAlgorithm();
25     return 0;
26 }
27 void bankersAlgorithm() {
28     int finished[MAX] = {0};
29     int safeSequence[MAX];
30     int count = 0;
31     // run till we cross total processes
32     while (count < numProcesses) {
33         int found = 0;
34         // run for every process
35         for (int i = 0; i < numProcesses; i++) {
36             if (finished[i] == 0) {
37                 int canExecute = 1;
38                 // do we have enough resources to exectue this process
39                 for (int j = 0; j < numResources; j++) {
40                     if (need[i][j] > available[j]) {
41                         canExecute = 0;
42                         break;
43                     }
44                 }
45                 // yes we can
46                 if (canExecute == 1) {
47                     for (int j = 0; j < numResources; j++) {
48                         available[j] += allocation[i][j];
49                     }
50
51                     safeSequence[count++] = i;
52                     finished[i] = 1;
53                     found = 1;
54                 }
55             }
56         }
57         // not a single process can execute
58         if (found == 0) {
59             printf("\nSystem is NOT in a safe state (Deadlock possible)\n");
60             return;
61         }
62     }
63     // while loop ends
64     printf("\nSystem is in SAFE state\n");
65     printf("Safe sequence: ");
66
67     for (int i = 0; i < numProcesses; i++) {
68         printf("P%d ", safeSequence[i]);
69     }
70 }
71 void input() {
72     printf("Enter number of processes: ");
73     scanf("%d", &numProcesses);
74
75     printf("Enter number of resources: ");
76     scanf("%d", &numResources);
77

```

```

78     printf("Enter Max matrix:\n");
79     for (int i = 0; i < numProcesses; i++) {
80         for (int j = 0; j < numResources; j++) {
81             scanf("%d", &max[i][j]);
82         }
83     }
84
85     printf("Enter Allocation matrix:\n");
86     for (int i = 0; i < numProcesses; i++) {
87         for (int j = 0; j < numResources; j++) {
88             scanf("%d", &allocation[i][j]);
89         }
90     }
91
92     printf("Enter Available resources:\n");
93     for (int j = 0; j < numResources; j++) {
94         scanf("%d", &available[j]);
95     }
96 }
97
98 void calculateNeed() {
99     for (int i = 0; i < numProcesses; i++) {
100         for (int j = 0; j < numResources; j++) {
101             need[i][j] = max[i][j] - allocation[i][j];
102         }
103     }
104 }
105
106 void display() {
107     printf("\nProcess\tAllocation\tMax\tNeed\n");
108
109     for (int i = 0; i < numProcesses; i++) {
110         printf("P%d\t", i);
111
112         for (int j = 0; j < numResources; j++) printf("%d ", allocation[i][j]);
113
114         printf("\t\t");
115
116         for (int j = 0; j < numResources; j++) printf("%d ", max[i][j]);
117
118         printf("\t");
119
120         for (int j = 0; j < numResources; j++) printf("%d ", need[i][j]);
121
122         printf("\n");
123     }
124
125     printf("Available: ");
126     for (int j = 0; j < numResources; j++) printf("%d ", available[j]);
127
128     printf("\n");
129 }

```